

TDS200 · Kryssplattform

Session 1

Introduction of Cross-Platform Mobile Development

Overview · Exam · Cross-platform · Environment setup · 3-tabs layout

Cross-Platform Mobile Development · Lecturer: Yuan Lin

Yuan Lin

- Yuan Lin, Associate Professor at Kristiania. I am based at Bergen campus.
- I have a Ph.D degree from NTNU.
- I have 10+ years industry experience in telecom and banking sector.
- I teach different frontend courses including Webutvikling, Cross-platform, Object-Oriented Programming, Database.
- I am also course responsible for Cross-platform and iOS programming.

For any questions regarding the courses, please contact me

Yuan.Lin@Kristiania.no

97438453

What is Cross -platform development?

- Build one application that can run on more than one operating system. i.e., iOS, Android.
 - Its is different from Native development, since Native builds the iOS app in Swift or Android app in Kotlin.
 - We use React Native to build code for both.

THE MAIN IDEA

**One shared codebase
→ multiple platforms**

Platforms

iOS • Android • Web • Desktop

Common technologies

React Native • Flutter
.NET MAUI • Capacitor/Cordova

Why use it?

✓ Faster development and easier maintenance
! Some platform-specific code may still be needed

SESSION 1 • STRUCTURE

How today's 2 hours run

0:00-0:20	Course overview The learning outcome, the 12 sessions, the exam, the tech stack
0:20-0:45	Theory: the cross-platform landscape Cross-platform vs Native
0:45-1:00	Break —
1:00-1:40	Live: setup & first app Toolchain, simulators, run the first app
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

What this course is

- **Build one real app, incrementally.**
 - Throughout the course, you will build a cross-platform photo/post-sharing app called **Postify**. Every session introduces new features and concepts.
- **Balance of theory and practice.**
 - Concepts, tech stack and code blocks. Examples are given by live coding.
 - Weekly assignment.
- **Outcome.**
 - A complete, authenticated, cloud-backed, and tested app — your exam baseline.
- **Syllabus.**
 - Your syllabus will be a combination of slides, codebase, framework documentation, and online materials.



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Learning outcomes

- Understand principles for developing applications that run on multiple mobile platforms.
- Be familiar with relevant development tools, frameworks, and design standards for major mobile platforms.
- Be able to explain the advantages and disadvantages of native development versus solutions publishable across multiple platforms.
- Be able to analyze requirement specifications and propose appropriate approaches for cross-platform development.
- Be able to develop cross-platform applications that integrate with suitable backend solutions and implement security in solutions.

<https://www.kristiania.no/studieportal/school-of-economics-innovation-and-technology/bachelorniva/tds200/kryssplattform/>



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Course setup

The codebase and learning materials will be uploaded to Canvas and Github.

- Canvas contains lecture slides, codebase in zipped format, assignments, and other learning resources.
- Github contains the codebase. The repository uses one branch per session.
- Students should follow the branches sequentially as the course progresses.



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Exam

- 3 weeks home exam – individual or in group
- Both technical and theoretical competence will be evaluated.

symbol	betegnelse	generell, ikke fagspesifikk beskrivelse av vurderingskriterier	Kommentarer om nivået
A	fremragende	Fremragende prestasjon som klart utmerker seg. Kandidaten viser svært god vurderingsevne og stor grad av selvstendighet.	Høy grad av måloppnåelse
B	meget god	Meget god prestasjon. Kandidaten viser meget god vurderingsevne og selvstendighet.	
C	god	Jevnt god prestasjon som er tilfredsstillende på de fleste områder. Kandidaten viser god vurderingsevne og selvstendighet på de viktigste områdene.	Middels grad av måloppnåelse
D	nokså god	En akseptabel prestasjon med noen vesentlige mangler. Kandidaten viser en viss grad av vurderingsevne og selvstendighet.	
E	tilstrekkelig	Prestasjonen tilfredsstiller minimumskravene, men heller ikke mer. Kandidaten viser liten vurderingsevne og selvstendighet.	Lav grad av måloppnåelse
F	ikke bestått	Prestasjon som ikke tilfredsstiller de faglige minimumskravene. Kandidaten viser både manglende vurderingsevne og selvstendighet.	



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Exam

- Your submission follows the structure:

TDS200_h26_[candidate-number].zip

Your candidate number must appear on the submission

/app — runnable product

- Must run in web and one of the emulators
- Source code + short readme included

50%

Portfolio.pdf

- Design, tech-stack choice, implementation, test & validation
- A dedicated reflection chapter
- AI usage documented together with the reflection

30%

Theory.pdf

- Report answering the theory chapter in the exam paper

10%

Demo.mp4

- Author navigates and explains the app (up to 5 min)

10%



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Exam

- **Your Portfolio.pdf should contain following elements:**
 - Design, tech stack choice, implementation overview, test % validation - 15%
 - A dedicated reflection chapter -10%

The reflections should demonstrate critical consideration of the student's own problem-solving process and may address questions like:

- What was the main problem or challenge in the activity?
 - How was the problem solved? Which technical or methodological decisions were made, and why?
 - What worked well and what could have been improved?
- AI usage documentation -5%



12 sessions × 2 hours

Lecture + live coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

The 12 sessions

1 Introduction & Foundations

2 Components & Navigation (Expo Router)

3 Components, State & Local Storage

4 Forms, Modals & Creating Data

5 Firebase Firestore CRUD

6 User Firestore CRUD & Local Images

7 Firebase Storage, Camera & Image Picker

8 Firebase Authentication & Context

9 Google sign-in, Firebase Rules & Guards

10 NativeWind & Geolocation

11 Google Maps & Queries

12 Exam & Wrap-Up

The technology stack



TypeScript
Typed JavaScript



Expo (SDK 52+)
Tooling & SDK



React Native
Native UI from JS



Expo Router
File-based nav



NativeWind
Tailwind styling



React Query
Server state & cache



Firebase
Auth · DB · Storage



Jest
Automated testing

NOTE

Firebase is the default backend; But you can also self-study Supabase (Postgres) as an alternative.

What you'll learn today

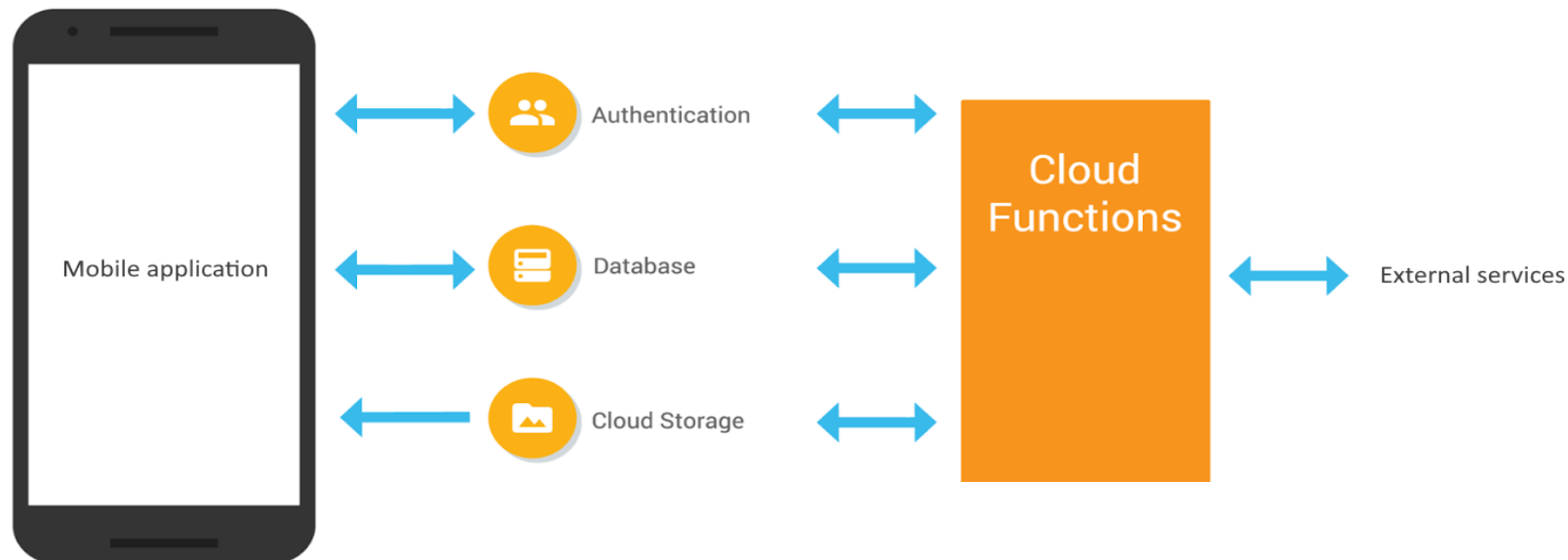
- Why cross-platform development exists at all
- How mobile apps render and run
- Why this course picks React Native + Expo
- Configure the environment and run your first app on a web and one simulator

Why cross-platform exists

- **Two dominant, incompatible platforms.**
 - iOS (Swift/Obj-C, Xcode, UIKit/SwiftUI) and Android (Kotlin/Java, Android Studio, Jetpack) have different languages, tools and UI paradigms.
- **Building natively means doing everything twice.**
 - Two codebases, two skill sets, duplicated logic, divergent bugs — roughly double the cost and time.
- **The business pressure.**
 - Reach both app stores from one codebase: faster time-to-market, smaller teams, a single source of truth.

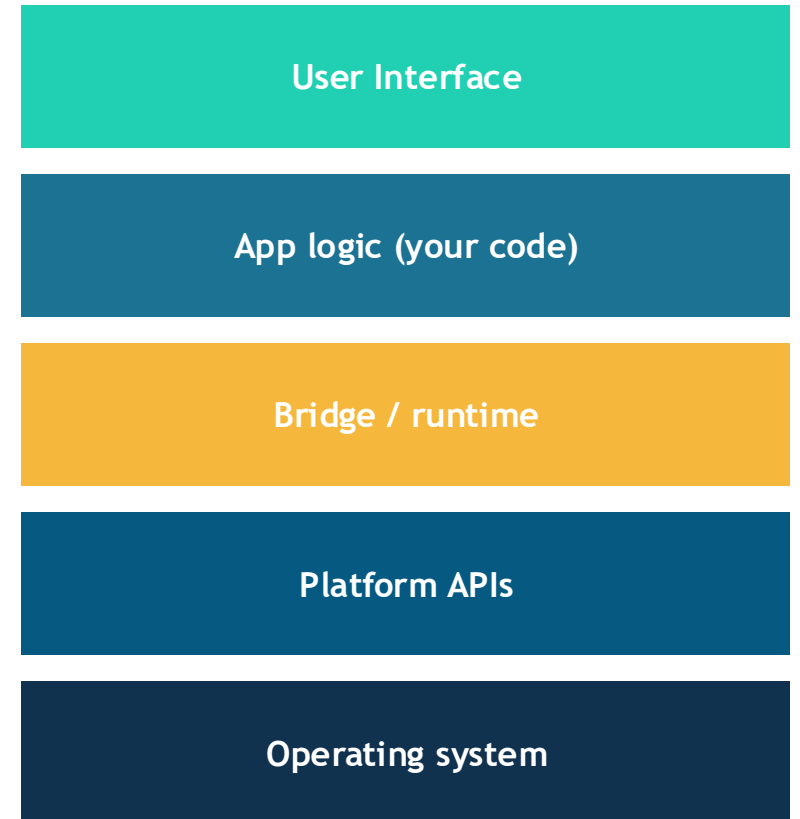
Mobile app overview

- **Mobile app:** user interface and interaction layer running on iOS and Android devices.
- **Cloud platform /Backend-as-a-Service (BaaS):** Cloud platforms such as Firebase or Supabase provide managed backend infrastructure where authentication database, cloud storage and serverless functions are hosted and maintained.
- **External services:** third-party systems integrated into backend or frontend
- **Deployment:** responsible for distributing, hosting, updating and scaling both mobile apps and backend services across app stores and cloud infrastructure.



How a mobile app is built

- Think in layers: UI layer → app logic → platform APIs → OS (iOS/Android).
- How is the UI rendered?
 - WebView rendering: UI runs inside an embedded browser · HTML / CSS rendered in a WebView · Additional abstraction layer · Examples (Ionic / Cordova / Capacitor)
 - Custom rendering engine: framework draws UI itself · independent of native UI toolkits · consistent appearance across platforms · Examples (Flutter)
 - Native rendering: uses real platform UI components · Examples (iOS: UIKit / SwiftUI · Android: Views /Jetpack Compose)
- Where the application logic executes?
 - Interpreted / JIT-compiled inside a VM or JS engine · Examples (React Native)
 - Compiled to native machine code · Examples (Flutter) · No JS runtime needed
 - Native execution · Examples (Swift / Kotlin) · Runs directly as native machine code
- How native APIs are accessed?
 - JS communicates with native APIs through bridge / interface -based communication · Examples (React Native)
 - Native functions exposed through plugins · Examples (Cordova / Capacitor)
 - Direct native access to platform SDKs and APIs · Examples (native development)



How we'll test

E2E (Detox, Maestro, Appium etc) — Actual user interaction on real device/emulator

Integration — Limited integration between components, state, hooks, APIs (still inside Jest)

Component — Testing rendered components with React Testing Library

Unit — Testing functions, utilities, hooks, components in isolation (Jest)

- Tooling: Jest (jest-expo) + React Native Testing Library.
- Mock Firebase and the AI call — never hit the network in a unit test.
- Test like a user: query by text/role and fire events, not internals.
- A detailed intro in Session 10. **We focus on Jest.**

NOTICE

Jest tests run in Node — passing (green) tests don't guarantee that the application works correctly on web, emulator, or real devices.

Why React Native + Expo for this course

- **React Native** is a framework for building mobile apps for both iOS and Android using JavaScript or TypeScript and React.
- **Expo** is a set of tools built around React Native. It simplifies project setup, testing, development and access to native device features. It starts with Metro bundler providing fast code loading and updates, and opens the app in iOS simulator, Android emulator, or Expo Go on a phone.
- We use **React Native + Expo** because it allows us to
 - Build iOS and Android apps from one codebase
 - Start projects quickly.
 - Test apps easily on a simulator or physical device
 - Focus on learning mobile development instead of complicated native setup.

OB

LIVE

Environment & simulator setup



Install the toolchain

1. Install Node.js (LTS ≥ 20) and npm.

- Verify with `node -v` and `npm -v`.

2. Install VS Code + extensions.

- ESLint, Prettier, React Native Tools.

3. Install Xcode. It takes time to install Xcode if using public wi-fi at school. So better to do it in advance

4. Install iOS simulator:

- Open Xcode → Settings. Go to Components or Platforms, select a recent iOS version, and click Download.
- Once it is installed, open Xcode → Open Developer Tool → Simulator.

Read “[Step-by-Step guide to run your first app](#)” (It is under SlideSerier canvas module)

terminal – create & run

```
node -v          # v20+
npm -v

# Create a TypeScript Expo app
npx create-expo-app@latest myapp
cd myapp

npx expo start

# i = iOS sim    a = Android
# w = web       or scan QR in Expo Go
```

Install the Expo Go app on your phone only if you want to test in on physical device

Three ways to see your app



iOS Simulator (macOS)

Install Xcode, open it once to accept the license, then press 'i'. Mac only.



Android Emulator

Android Studio → SDK + an AVD (e.g. Pixel 7, API 34). Start it, press 'a'. Any OS.



Real device (Expo Go)

Scan the QR with Expo Go. Phone and computer on the same Wi-Fi.

TIP

No Mac? The Android Emulator or a real phone with Expo Go cover everything in this course.

Your first screen

- A screen is a component that returns JSX.
- View lays out; Text renders text.
- Styles are objects via StyleSheet.create().
- Save — Fast Refresh updates instantly.

app/index.tsx

```
import { View, Text, StyleSheet } from 'react-native';

export default function Home() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, TDS200 🙌</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center',
               alignItems: 'center' },
  title: { fontSize: 24, fontWeight: '600' },
});
```

Stylesheet

- StyleSheet is React Native built-in styling system.
 - It defines how components look and are arranged on the screen.
 - Similar to CSS; but written as JavaScript objects.
 - Helps separate presentation from application logic.
 - Makes styles reusable and easier to maintain.
 - StyleSheet definitions is usually located in the end of a React Native file.

●●● Card.tsx

```
const styles = StyleSheet.create({  
  box: { padding: 16, gap: 8 },  
  title: { color: "#fff", fontSize: 20 },  
});
```

```
<View style={styles.box}>  
  <Text style={styles.title}>Hi</Text>  
</View>
```

KEY POINTS

- create() groups related styles
- reference with styles.name
- keys are camelCased CSS-like props

Your first app with simple Tab navigator

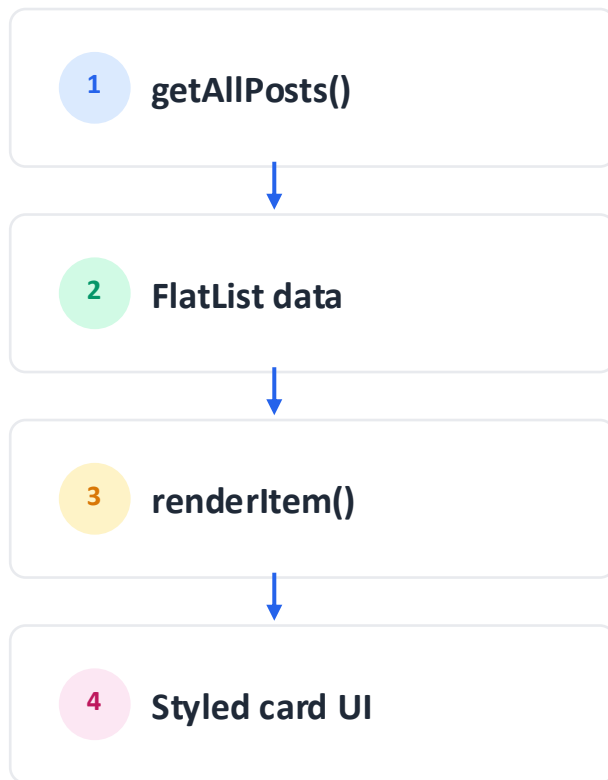
- The Tab navigator provides a set of tabs at the bottom of the screen.
- Expo Router automatically creates a tab for each screen defined inside the `<Tabs>` component.
 - `Index.tsx` -> Home tab. `Index.tsx` is always the default route for a folder. When the app starts, Expo Router navigates to `/`, which corresponds to `index.tsx`.
 - `authenticationPage` -> Sign-in tab.
- Each `<Tabs.Screen>` configures:
 - `name` – the route or screen file
 - `Title` – the label shown in the tab bar
 - `tabBarIcon` – an icon displayed alongside the tab label.

```
export default function RootLayout() {
  return (
    <Tabs>
      <Tabs.Screen
        name="index"
        options={{
          title: "Home",
          tabBarIcon: ({ color }: { color: string }) => (
            <AntDesign name="home" size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="authenticationPage"
        options={{
          title: "Sign in",
          tabBarIcon: ({ color }: { color: string }) => (
            <AntDesign name="login" size={24} color={color} />
          ),
        }}
      />
    </Tabs>
  );
}
```

Home Screen with FlatList

- FlatList is React Native core component.

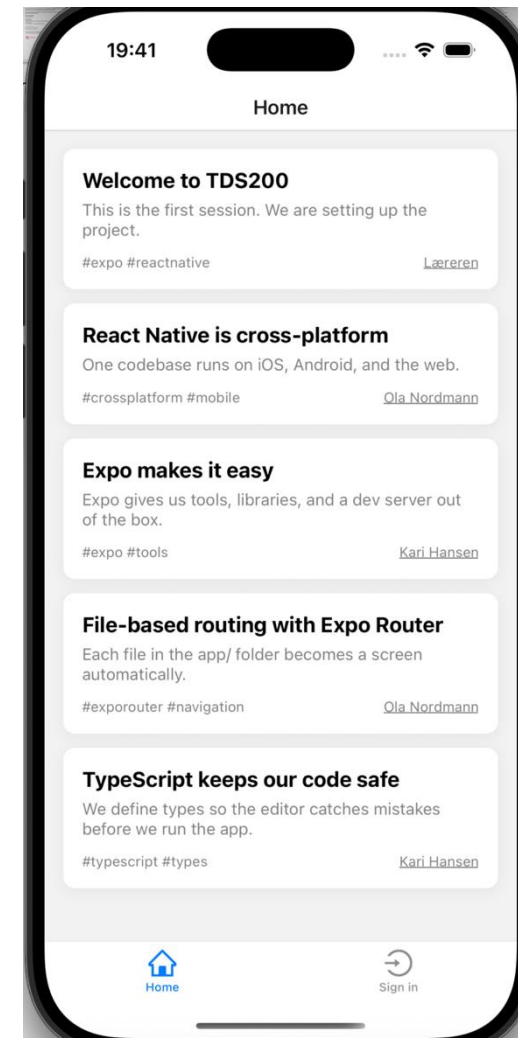
Data flow



Core idea

```
const posts: PostData[] = getAllPosts();

<FlatList
  data={posts}
  keyExtractor={({item}) => item.id}
  renderItem={({item}) => (
    <View style={styles.card}>
      <Text>{item.title}</Text>
      <Text>{item.description}</Text>
    </View>
  )}
/>
```



LIVE • FIRST APP

FlatList props to explain

FlatList props to explain

data The array that FlatList renders

keyExtractor Unique id for each row

renderItem How one post becomes UI

ItemSeparator Reusable spacing between rows

contentContainerStyle Padding around the whole list

ASSIGNMENT 1



Hello, cross-platform

Get your environment working, run an app on a simulator, and reason about the landscape.

YOUR TASKS

- Install Node, VS Code and Expo Go; create a new Expo app.
- Run it on an iOS Simulator, Android Emulator, OR a real device.
- The app already has Tab navigation. Your task is to add one new tab called “Profile”. On the “Profile” page, customize the screen by adding fields such as name, image, biology, etc. Also style the page using React Native components, for example, use “View”, “Text”, “Image”, “StyleSheet”.

DELIVERABLE

A screenshot of your running app + a short reflection (PDF or Markdown).

EST. EFFORT

≈ 2 hours